

11 — Economics

Objetivo: entender el **costo real** de generar una app con Puglit — de punta a punta y sin marketing. Como el default corre 100% local sobre Ollama, no hay costo de tokens: el costo es **GPU-time**. Esta sección modela ese costo desde el trace real de cada build (`web/lib/run-trace.ts` + `web/lib/swarm-profile.ts`), lo compara contra correr los mismos modelos por API (RunPod GPUs vs SiliconFlow), y proyecta el costo por proyecto, por usuario y a escala (1 / 100 / 1.000 / 10.000 usuarios). Todo verificado contra el código en `/Users/alvaroz/projects/2026/puglit` (`web/lib/openai.ts` , `web/lib/swarm-profile.ts` , `web/lib/run-trace.ts` , `infra/`).

1. Cost Model

El modelo de costos de Puglit nace de una sola decisión de arquitectura: **un único chokepoint** para toda inferencia, `call()` en `web/lib/openai.ts` , con un proveedor por defecto que es **local**. Cuando nada está configurado, `DEFAULT_PROVIDER` resuelve a `ollama` y los modelos por defecto son open-weight:

```
// web/lib/openai.ts — providerModels("ollama")
{ premium: "gemma2:27b", balanced: "gemma2", cheap: "gemma2:2b", code: "qwen2.5-coder:7b" }
```

En producción la caja de referencia sube esos lanes a **qwen2.5-coder:32B** (code/premium/judge) + **gemma2:9B** (balanced/cheap), según `infra/setup-gpu-box.sh` . La consecuencia económica es directa:

Local Ollama = \$0 por token. No hay metering, no hay factura por uso, no hay rate-limit pago. El único costo es el **tiempo de GPU** durante el cual el pod está encendido.

Esto invierte la unidad de costo respecto de cualquier stack basado en API. En un sistema API-first, el costo es **tokens × precio/token** y crece linealmente con cada llamada. En Puglit, el costo es **horas-de-GPU × precio/hora** y es **independiente del número de tokens** — un build que dispara 80 llamadas LLM cuesta lo mismo por minuto que uno que dispara 8; lo único que importa es **cuánto tarda** el pod en estar prendido.

La fórmula base

```
costo_build = (latencia_total_del_build / 3600 s) × precio_GPU_por_hora
costo_idle  = (tiempo_pod_encendido_sin_build / 3600 s) × precio_GPU_por_hora
costo_total = costo_build + costo_idle
```

Donde `latencia_total_del_build` es exactamente lo que mide el run-trace: la suma de `ms` de todas las llamadas del build más el tiempo determinístico (módulos, runtime gate, vitest). El patrón operativo recomendado en `setup-gpu-box.sh` — **encender** → **trabajar** → **STOP** — existe precisamente para llevar

`costo_idle` a cero: el volume disk persiste los ~77GB de modelos, así que apagar el pod entre sesiones no cuesta re-descargas.

Por qué "cost" en swarm-profile no es dinero

`web/lib/swarm-profile.ts` puntúa cada build en cuatro categorías (Cost / Latency / Reliability / Context). Su comentario de cabecera lo dice sin vueltas:

```
// On a local A40 the $ is ~0, so "cost" is wasted GPU-time: a premium model on a
// trivial step, or the same prompt sent twice.
```

O sea: en el modelo local, "cost" = **GPU-time desperdiciado**, no dólares. Un modelo premium en un paso trivial o un prompt mandado dos veces no agregan a una factura de tokens (no hay), pero **sí queman segundos de GPU** que se traducen en horas-de-pod. Por eso la categoría Cost y la categoría Latency están tan acopladas, y por eso optimizar el run (sección 10) reduce literalmente el costo en dólares vía menos tiempo de pod.

2. Token Consumption

Aunque local no se factura por token, el sistema **igual los cuenta**, porque los tokens son el predictor de latencia y la señal que alimenta los audits. El collector es minúsculo y sin I/O (`web/lib/run-trace.ts`):

```
export type Span = { tier: string; model: string; promptChars: number; outChars: number; ms: number;
```

Cada `call()` registra **un span** en el `finally` (nunca rompe la llamada por instrumentarla). `swarm-profile.ts` convierte chars a tokens con una heurística estándar:

```
const TOK = (chars: number) => Math.ceil(chars / 4) // ~4 chars por token
```

Y agrega por run:

```
tokensIn  = Σ TOK(promptChars)    // contexto enviado
tokensOut = Σ TOK(outChars)       // generación
byTier    = { premium, balanced, cheap, code, other } // conteo de llamadas por lane
```

Mapa de consumo por etapa del pipeline

El pipeline `buildAdvance` (`web/lib/app-builder.ts`) reparte el consumo de forma muy desigual entre lanes. El **lane code** (qwen2.5-coder) domina el `tokensOut` porque genera archivos enteros; el **lane premium** domina el `tokensIn` porque el blueprint, el torneo y la review adversarial cargan mucho contexto.

Etapa	Lane dominante	Característica de tokens	Por qué
plan / blueprint	premium	<code>tokensIn</code> alto, <code>tokensOut</code> medio	Carga idea + playbook ARCHITECT + catálogo de módulos
genetic tournament (3 teams + jurados)	premium	<code>tokensIn</code> MUY alto (prompts con prefijo común)	3 blueprints divergentes + panel multi-juez sobre el mismo contexto
critique / brief	balanced	medio/medio	Specs estructuradas, JSON-mode
routes / pages	code	<code>tokensOut</code> MUY alto	Genera archivos completos (handlers, páginas, SQL)
finalize (módulos)	— (determinístico)	0 tokens	<code>deterministicX(config, bp)</code> inyecta archivos sin LLM
adversarial review	premium	<code>tokensIn</code> alto	3 lentes ortogonales sobre el árbol generado
swarm-repair	code	medio	Repara phantom-tables, escalada a frontier

Insight clave: el `finalize` con inyección determinística de los ~85 módulos (`web/lib/module-registry.ts`) consume **cero tokens**. Toda capacidad que se mueve de "el LLM lo escribe" a "el módulo lo inyecta" sale gratis y, además, baja la latencia del build. Esto es economía de diseño, no de tuning.

Lo que el trace NO captura (honesto)

`TOK = chars/4` es una aproximación; el tokenizer real de cada modelo difiere. El span no guarda el tokenizado exacto del provider (no lo necesita: su trabajo es rankear desperdicio, no facturar). Cuando se usa un provider API que **sí** factura, el conteo verdadero viene del `usage` de la respuesta del provider, no del trace. El trace es para optimización local; el billing API es del provider.

3. Model Cost Analysis — Local vs API

Hay dos formas de pagar por la inteligencia del swarm:

1. **Local (GPU-time):** alquilás una GPU por hora, corrés Ollama, los tokens son gratis.
2. **API (per-token):** apuntás un tier a un endpoint OpenAI-compatible (vía `PUGLIT_*_BASE_URL` / `PUGLIT_*_API_KEY`) y pagás por token consumido.

3.1 — RunPod GPUs (modelo local)

La caja de referencia es un **A40 48GB a \$0.44/hr**. El factor limitante es la **VRAM**: qwen2.5-coder:32B @ Q4 ocupa ~20GB y necesita headroom para gemma2 + embeddings + KV cache. Estas son las opciones reales y su trade-off:

GPU	VRAM	\$/hr	Aguanta 32B @ Q4	Headroom / notas
A40 (referencia)	48GB	\$0.44	Sí (~20GB)	Mejor \$/VRAM para el stack; KV cache + gemma2 + embeddings entran cómodos
L40S	48GB	\$0.99	Sí	Más rápido por token que el A40 (Ada Lovelace), pero 2,25× el precio
RTX PRO 6000	96GB	\$2.09	Sí, holgado	Permite un 70B Q4 o varios modelos residentes a la vez (sin swap)
H200	141GB	\$4.39	Sí, holgado	Frontier-throughput; sólo se justifica para batch masivo (brain-training 100-build)

Los modelos pesan **~77GB en disco** (qwen2.5-coder:32B ~20GB + gemma2:9B + nomic-embed-text + opcional visión). El volume persistente (100GB recomendado) evita re-descargarlos en cada arranque — costo de storage aparte del compute.

Costo efectivo del tier code/premium = el precio/hora de la GPU, prorrateado por minuto de uso. Con A40 a \$0.44/hr, un minuto de GPU activa cuesta **\$0.0073**. Esa es la unidad que multiplica toda la sección de build.

3.2 — SiliconFlow API (modelo cloud, per-token)

Para tiers livianos en picos, o para acceder a modelos **más grandes** de los que entran en el A40, se puede apuntar un tier a la **SiliconFlow API** (OpenAI-compatible, drop-in en `resolve(model)` sin tocar código). Los modelos cloud relevantes:

Modelo (SiliconFlow)	Clase	Uso recomendado en Puglit	Comentario
GLM 5.2	Frontier-ish open	premium / judge en picos	Por encima de qwen2.5-coder:32B local en razonamiento
DeepSeek V4	Code/reasoning grande	lane code pesado	Mejor codegen que el 32B local; ideal para apps complejas

El wiring es por env, sin cambios de código (la abstracción de proveedor ya lee estos overrides):

```
# codegen pesado → modelo grande cloud, con A40 local como fallback confiable
PUGLIT_CODE_BASE_URL=https://api.siliconflow.cn/v1
PUGLIT_CODE_API_KEY=sk-...      # model: "deepseek-v4"
```

3.3 — La comparación que importa

La pregunta no es "¿cuál modelo es mejor?" sino "¿a qué volumen conviene cada modelo de costo?". Local tiene **costo fijo por hora** (independiente de tokens); API tiene **costo variable por token** (independiente del tiempo). El cruce depende de **cuántos builds por hora** exprimís de la GPU:

Builds/hora exprimidos del A40	Costo GPU por build (A40 \$0.44/hr)	¿API más barata?
1 build/hora (ocioso entre medio)	~\$0.44/build	A menudo sí (API sólo paga lo que usás)
4 builds/hora	~\$0.11/build	Empieza a ganar local
8 builds/hora (pipeline saturado)	~\$0.055/build	Local gana cómodo

Regla práctica: API conviene a **bajo volumen / picos / modelos que no entran en la GPU**; local conviene a **volumen sostenido** (donde el costo fijo por hora se diluye entre muchos builds). Por eso la recomendación de infra es híbrida: **A40 local como caballo de batalla y fallback confiable**, SiliconFlow para picos o para subir el lane code a DeepSeek V4 cuando la app lo amerita. El judge/critic se mantiene en el modelo local **consistente** — no se lo deja rotar entre providers a mitad del scoring (rompería la comparabilidad del torneo).

4. Build Cost Analysis

El costo de un build es **latencia × precio-GPU**. La latencia la mide el run-trace; el precio-GPU sale de la sección 3. Modelemos un build representativo.

4.1 — Anatomía de latencia de un build

Un build full-stack típico (estilo Stayforge: ~80 archivos, 8 tablas) dispara del orden de **40–80 llamadas LLM** repartidas entre lanes, más etapas determinísticas (módulos, runtime gate, vitest). Sobre la caja de referencia (A40, qwen2.5-coder:32B local), los tiempos por etapa son aproximadamente:

Etapas	Llamadas LLM aprox.	Latencia aprox.	Lane
plan + blueprint	1–2	0,5–2 min	premium
genetic tournament (3 teams + jurados)	6–12	3–8 min	premium
critique + brief	3–6	1–3 min	balanced
routes + pages	20–40	6–15 min	code
finalize (módulos, determinístico)	0	<0,5 min	—
adversarial review	3–6	1–3 min	premium
runtime gate + vitest + Queen review	1–4	2–6 min	code/premium
Total build	~40–80	~15–35 min	mixto

El **tournament** (iteración 3: 3 teams divergen + panel multi-juez) y **routes/pages** son los dos sumideros de latencia. El finalize es gratis en tiempo y en tokens.

4.2 — Costo GPU por build, por GPU

Tomando un build de **~25 min** (punto medio de la tabla anterior) como referencia, el costo es lineal con el precio/hora:

GPU	\$/hr	Build 15 min	Build 25 min	Build 35 min
A40	\$0.44	\$0.11	\$0.18	\$0.26
L40S	\$0.99	\$0.25	\$0.41	\$0.58
RTX PRO 6000	\$2.09	\$0.52	\$0.87	\$1.22
H200	\$4.39	\$1.10	\$1.83	\$2.56

En el caso canónico — **A40, build de 25 min — un build full-stack completo cuesta ~\$0.18**. Eso incluye blueprint, torneo genético de 3 equipos, generación de ~80 archivos, review adversarial de 3 lentes, runtime gate que bootea la app, y vitest con coverage. Comparado con el costo humano-equivalente (días de un dev), el costo marginal en GPU es despreciable; el costo que importa es el **idle**, no el build.

4.3 — El multiplicador escondido: el torneo

El torneo genético corre **3 equipos en paralelo conceptual** (Lean / Enterprise / Hacker) más un panel de jueces. Si los 3 corren **secuencialmente** en una sola GPU, la latencia del blueprint se ~triplica. Acá es donde una GPU más grande (RTX PRO 6000 96GB, varios modelos residentes) o SiliconFlow (paralelismo real sin VRAM compartida) pueden **bajar el wall-clock** del build aunque suban el \$/hr — y, como el costo es latencia×precio, a veces la GPU más cara sale **igual o más barata por build** si recorta minutos. Es exactamente el trade-off que swarm-profile expone con el audit `duplicate-calls` (latencia) y que LMCache ataca (KV-cache de los prompts con prefijo común del panel de jurados).

5. Infrastructure Cost

Más allá del compute de GPU, la plataforma tiene costos de soporte. Casi todos son **fijos y chicos** porque el stack core corre **nativo en la caja** (Ollama + Postgres + Next), y sólo los gateways opcionales corren en Docker (`infra/setup-gateways.sh`).

Componente	Modelo de costo	Estimado	Notas
GPU compute (A40)	\$/hr mientras encendido	\$0.44/hr	El grueso. Se apaga entre sesiones (idle→\$0)
Volume / disco persistente	\$/GB-mes	~\$0.10/GB-mes típico RunPod → ~\$10/mes por 100GB	Persiste los ~77GB de modelos + Postgres + builds
Postgres 14 + pgvector	Incluido en la caja	\$0 extra	Corre nativo, mismo host
Sidecars Python (scrapegraph, scraper-server)	Incluido (nohup en la caja)	\$0 extra	Comparten CPU/RAM del host
Gateways Docker (MinIO, Meilisearch, apprise, n8n, Nango, freellmapi)	Incluido (mismo host, ~40MB-RAM c/u idle)	\$0 extra	Sólo los que el usuario levante; opcionales
Egress / red	\$/GB	Marginal	Pull de modelos (una vez), git/bucket snapshots del cerebro

Storage es el único costo verdaderamente "siempre encendido". El compute se apaga; los modelos en el volume no. ~\$10/mes de volume es el piso de costo de tener la plataforma "lista para encender". Los snapshots del cerebro a git/bucket ([infra/brain-snapshot.sh](#)) son baratos (texto + embeddings comprimidos) y permiten **destruir el pod entero** y restaurarlo después — bajando incluso ese piso si se vive de snapshots.

Costo mensual de plataforma (3 perfiles de uso)

Perfil	GPU horas/mes	Compute (A40)	Volume	Total/mes
Hobby (enciende a demanda, ~20h/mes)	20	\$8.80	\$10	~\$19
Activo (~4h/día, 120h/mes)	120	\$52.80	\$10	~\$63
Siempre prendido (730h/mes)	730	\$321	\$10	~\$331

El salto de "siempre prendido" a "encender a demanda" es **17x** en costo. El patrón STOP del pod no es una sugerencia de estilo: es la palanca económica más grande de toda la plataforma.

6. Gateway Cost

Los gateways son **sidecars de capacidad** que dan poderes a los módulos inyectados en las apps generadas (scraping, search, storage, notifications, OAuth, workflows). Económicamente tienen tres propiedades buenas:

- 1. **Corren en el mismo host** sobre la red bridge `puglit-net` → **cero costo de compute adicional** (comparten la CPU/RAM de la caja que ya pagás por la GPU).
- 2. Son **opcionales y bajo demanda** → sólo pagás (en RAM) los que levantás.
- 3. Varios usan **tier gratis** del proveedor upstream o corren **100% local**.

Gateway	Puerto	Costo marginal	Naturaleza
MinIO (storage S3-compat)	9000/01	\$0 (local)	Disco del host
Meilisearch (search)	7700	\$0 (local)	RAM del host
apprise (notifications)	8000	\$0 (local)	Puente a canales del usuario
n8n (workflows)	5678	\$0 (local)	Orquestación local
Nango (OAuth)	3003	\$0 (local)	Broker de tokens
scrapegraph / scraper-server	5055 / 8200	\$0 (usa Ollama local)	Inferencia local, sin API
freellmapi (LLM free-tier proxy)	3001 / 5173	\$0 + tier gratis upstream	~40MB RAM idle; agrega 16+ providers free

El gateway con mejor economía es **freellmapi**: ~40MB de RAM idle, y desbloquea acceso **gratis** a modelos más grandes que el A40 (Qwen3-235B, DeepSeek V4, GLM-4.7, Groq Llama 4). El caveat es honesto y está documentado: **sin frontier y sin SLA**, degrada cuando los free-tiers pegan su cap diario. La estrategia correcta lo usa para **volumen** con el A40 local como **fallback confiable**, y nunca para el judge (que debe ser consistente). Donde SiliconFlow es el camino **pago-pero-estable**, freellmapi es el **gratis-pero-best-effort** — ambos drop-in por env.

7. Cost Per Project

"Proyecto" = un build completo y entregable. El costo por proyecto es el costo de build (sección 4) más la fracción de idle prorrateada y la fracción de storage del mes.

Costo marginal puro (sólo el compute del build)

GPU	Build 25 min	Costo marginal/proyecto
A40	\$0.18	\$0.18
L40S	\$0.41	\$0.41
RTX PRO 6000	\$0.87	\$0.87
H200	\$1.83	\$1.83

Costo totalmente cargado (incluye idle + storage amortizado)

El costo real por proyecto depende de **cuántos proyectos hacés por mes**, porque el idle y el storage son fijos y se reparten:

Proyectos/mes	Compute builds (A40, \$0.18 c/u)	Idle + storage (perfil Activo, ~\$63 - builds)	\$/proyecto cargado
1	\$0.18	~\$63	~\$63
10	\$1.80	~\$63	~\$6.30
50	\$9.00	~\$63	~\$1.26
200	\$36	~\$63	~\$0.31
1000	\$180 (necesita más horas/GPU)	~\$63+	~\$0.24

El costo marginal de un proyecto es **~\$0.18**. El costo **cargado** está dominado por los fijos hasta que llegás a volumen. La economía de Puglit recompensa fuertemente el **uso intenso de la caja prendida**: hacer 50 builds en una sesión de 4 horas amortiza el idle a casi nada. El peor caso económico es **prender el pod para un solo build** — ahí pagás overhead de arranque (carga de modelos a VRAM) por un único proyecto.

8. Cost Per User

En el caso default — **self-hosted, un usuario, su propia caja** — el "costo por usuario" ES el costo de plataforma de la sección 5: entre **~\$19/mes** (hobby) y **~\$331/mes** (siempre prendido). No hay costo por asiento ni licencia: Puglit es open-source y el usuario es dueño de su compute.

Para un **deploy multi-tenant** (un operador corre Puglit para varios usuarios sobre una sola caja), el costo por usuario es el costo de plataforma dividido por los usuarios activos, más el costo marginal de los builds que cada uno dispara:

$$\text{costo_por_usuario} = (\text{fijos_plataforma} / \text{usuarios_activos}) + (\text{builds_del_usuario} \times \$0.18)$$

Usuarios sobre 1 A40 "siempre prendido" (\$331/mes fijos)	Fijo/usuario	+ 5 builds/usuario	\$/usuario/mes
1	\$331	\$0.90	~\$332
10	\$33.10	\$0.90	~\$34
50	\$6.62	\$0.90	~\$7.52
200	\$1.66	\$0.90	~\$2.56

El costo por usuario **se desploma** con la densidad de tenants, hasta toparse con la **capacidad de la GPU** (una sola GPU serializa los builds; pasada cierta concurrencia hay que sumar GPUs — sección

9). El punto dulce: **decenas de usuarios sobre una A40** lleva el costo a **\$/usuario de un solo dígito de dólares por mes**, asumiendo que los builds no se solapan demasiado.

9. Scaling Scenarios

Modelamos 1 / 100 / 1.000 / 10.000 usuarios activos. La variable que fuerza el escalado **no es el almacenamiento ni los tokens** (gratis local) — es la **conurrencia de builds sobre la GPU**: una A40 corre **un build pesado a la vez** de forma cómoda (el 32B llena la VRAM). El throughput sostenido es del orden de **~2–4 builds/hora por GPU** (builds de 15–35 min).

Supuesto de carga: **5 builds/usuario/mes**, distribuidos. 1 GPU sirve **~1.500–2.900 builds/mes** a utilización razonable (no 100%, para no encolar).

Usuarios	Builds/mes (5 c/u)	GPUs A40 necesarias	Compute GPU/mes	Storage	Gateways	Total/mes	\$/usuario/mes
1	5	1 (a demanda, ~20h)	~\$9	\$10	\$0	~\$19	\$19
100	500	1 (parcial, ~250h)	~\$110	\$10	\$0	~\$120	\$1.20
1.000	5.000	2–3 (siempre prendidas)	~\$700–950	\$30	\$0	~\$730–980	~\$0.85
10.000	50.000	20–30 (fleet)	~\$7.000– 9.500	\$300	~\$50	~\$7.350– 9.850	~\$0.85

Lecturas del escalado: - De 1 → 100 usuarios el costo por usuario cae **~16x** (\$19 → \$1.20): se amortiza la única GPU que ya tenías. - De 100 → 10.000 el costo por usuario se **estabiliza** (~\$0.85): a partir de ahí el costo es ~lineal con los builds, porque cada GPU adicional sirve una cuota fija de builds. **No hay economía de escala mágica más allá del punto donde la GPU se satura** — pero tampoco hay deseconomía: el costo marginal por build se mantiene en ~\$0.18 sin importar la escala. - **El storage y los gateways son ruido** a cualquier escala (cero costo de tokens). El 95%+ del costo es **GPU compute**. - A 10.000 usuarios conviene **mezclar SiliconFlow** para absorber picos sin sobreaprovisionar la fleet (pagás per-token sólo en el pico en vez de tener GPUs ociosas para el peor caso).

Sensibilidad a la GPU elegida

A 1.000 usuarios, la elección de GPU mueve el total de forma fuerte — pero recordá que una GPU más rápida hace builds más cortos, así que el \$/build no escala 1:1 con el \$/hr:

GPU para la fleet (1.000 usuarios)	\$/hr	\$/build aprox.	Compute/mes aprox.
A40	\$0.44	\$0.18	~\$700–950
L40S (builds ~30% más rápidos)	\$0.99	~\$0.29	~\$1.150–1.450
RTX PRO 6000 (paraleliza el torneo)	\$2.09	~\$0.55	~\$2.200–2.750

La **A40 sigue siendo el óptimo de \$/build** salvo que el wall-clock por build sea un SLA duro (entonces L40S o paralelizar el torneo en una GPU grande valen la pena). El brain-training batch de 100 builds es el único caso donde una H200 se justifica: maximiza builds/hora para pre-cargar el cerebro rápido.

10. Optimization Opportunities

Como "cost" = GPU-time desperdiciado, **optimizar el run reduce dólares directamente** (menos minutos de pod). `swarm-profile.ts` no sólo puntúa: emite **fixes rankeados por ms ahorrados**. Los pesos del score son `cost 0.30 · latency 0.30 · reliability 0.20 · context 0.20` (**WEIGHTS**), y los cuatro audits atacan cuatro formas concretas de desperdicio:

10.1 — `model-tier-mismatch` (cost)

```
const TRIVIAL_TOK = 400
const tierMismatch = spans.filter(s => s.tier === "premium" && TOK(s.promptChars) < TRIVIAL_TOK)
savingsMs = Σ (ms × 0.5) // la mitad del tiempo de cada llamada mal-asignada
```

Detecta llamadas al lane **premium** cuyo prompt completo es **< 400 tokens** — un paso trivial no necesita el modelo caro/lento. El fix (hint literal del código): *"Usar `MODELS.balanced` / `cheap` para pasos triviales (prompt corto) en vez de premium."* En el modelo local, un premium innecesario es un 32B haciendo trabajo que un gemma2:2b resolvía en una fracción del tiempo → **GPU-time tirado**. Ahorro estimado: **50% del tiempo** de cada llamada mal-tier.

10.2 — `context-bloat` (context)

```
const CONTEXT_BLOAT_TOK = 12000 // override: PUGLIT_CONTEXT_BLOAT_TOK
const bloat = spans.filter(s => TOK(s.promptChars) > CONTEXT_BLOAT_TOK)
savingsMs = Σ (ms × 0.3)
```

Detecta llamadas con **> 12.000 tokens de prompt**. Mandar "todo el árbol" en vez de los archivos/tramos relevantes infla el contexto, alarga el tiempo de inferencia (más KV cache, más prefill) y degrada la calidad. Fix: *"Recortar el contexto: mandar solo los archivos/tramos relevantes, no todo."* Ahorro estimado: **30% del tiempo** de cada llamada inflada. Este audit es el que más se beneficia de LMCache (los prefijos comunes se cachean en vez de re-prefillear).

10.3 — `duplicate-calls` (latency)

```
// clave = tier + tamaño-de-prompt redondeado a 50 chars
const k = `${s.tier}:${Math.round(s.promptChars / 50)}`
// cada repetición de la misma clave cuenta como duplicado
savingsMs = dups × avgMs
```

Detecta el **mismo prompt mandado dos veces** (mismo tier + tamaño). Cada duplicado es un build pagando dos veces por el mismo resultado. Fix: *"Cachear/dedupe prompts idénticos repetidos en el run."* Ahorro estimado: **el tiempo promedio de una llamada × cada duplicado**. Es el audit con mayor upside cuando el torneo reusa contexto: el panel de jurados manda prompts con prefijo casi idéntico → candidatos perfectos para dedupe + KV-cache.

10.4 — errors-retries (reliability)

```
const errors = spans.filter(s => !s.ok).length
savingsMs = errors × avgMs
```

Cada llamada fallida (`ok: false`) gastó GPU-time sin producir resultado y disparó un reintento. Fix: *"Validar inputs antes de la llamada; bajar temperatura en JSON estructurado."* Acá conecta con el grammar-constrained decoding de `openai.ts` (Ollama native `format` + OpenAI `json_schema` con `strict: true`): forzar JSON válido por construcción elimina la clase entera de "el modelo devolvió markdown, parseo falló, reintento" → menos `errors` , menos GPU-time quemado.

10.5 — El loop económico completo

```
return `RUN SCORE ${rs.score}/100 · cost ${cost} · latency ${latency} · ... · fixes: ${top}`
```

`summarizeRun()` imprime el score y los **top-3 fixes con ms ahorrados** en el log del build. El número de "ms ahorrados" es dinero ahorrado en el modelo local: cada segundo recortado del build es un segundo menos de pod encendido. Y como el score retroalimenta la selección de agentes/teams (la señal vuelve al evolution engine), el sistema **aprende a ser más barato build a build** — no por un tuning manual, sino porque los teams que generan menos desperdicio ganan más torneos.

Audit	Categoría	Qué quema GPU-time	Ahorro modelado	Palanca de infra que ayuda
<code>model-tier-mismatch</code>	cost	32B en paso trivial	50% × ms	routing por tier estricto
<code>context-bloat</code>	context	prefill de >12k tok	30% × ms	LMCache (KV de prefijos)
<code>duplicate-calls</code>	latency	mismo prompt 2×	avg × dups	dedupe + KV-cache
<code>errors-retries</code>	reliability	llamada fallida + retry	avg × errors	grammar-constrained JSON

Conclusión económica: en Puglit el costo no se baja "comprando tokens más baratos" — se baja desperdiciando menos **GPU-time**, y el sistema **se mide y se corrige a sí mismo** para hacerlo. Sumá las tres palancas de mayor impacto — **STOP del pod** (sección 5, hasta 17× menos costo fijo), **densidad de tenants** (sección 8, costo/usuario de 3 dígitos a 1 dígito), y los **audits de swarm-profile**

(esta sección, menos minutos por build) — y el costo total de operar Puglit a escala converge a **~\$0.18 marginales por proyecto sobre una A40 de \$0.44/hr**: el costo de generar una app full-stack completa, testeada y self-healed, es literalmente el de unos minutos de una GPU de medio dólar la hora.